Lesson 7 Teacher's Key: On the Move

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

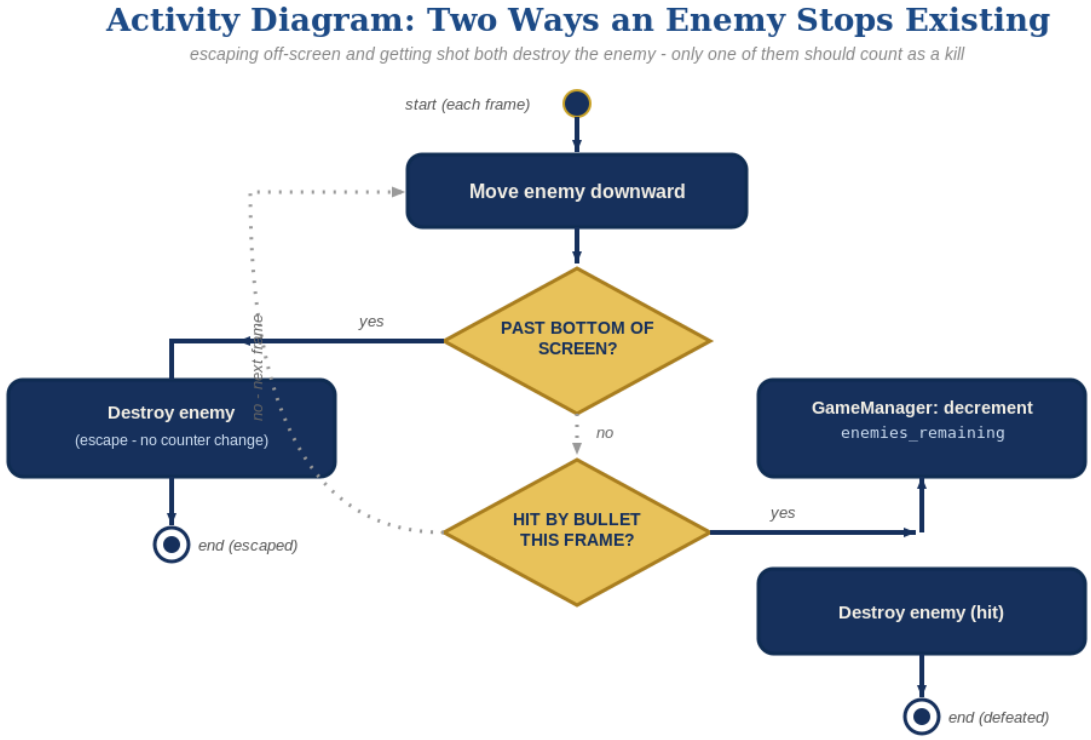
What This Lesson Teaches

The core lesson-7 bugfix: GameMaker's Destroy event fires for ANY instance_destroy() call regardless of cause, so it cannot be trusted for win-counting once a second destruction pathway (off-screen escape) exists. The fix moves the counter to the one event that actually represents a hit.

The Bug Being Fixed

This is the primary bugfix engine for this lesson, alongside 001 Game Creator. Lesson 6 wired enemies_remaining -= 1 into obj_enemy's Destroy event. That event fires on literally any instance_destroy() call against an obj_enemy instance - including the new escape-cleanup instance_destroy() this lesson adds inside obj_enemy's Step event. Left unfixed, an enemy escaping unshot would still decrement enemies_remaining and could trigger a false "You Win!". The fix: delete the Destroy event's logic entirely, and add the decrement (plus the win check) into obj_bullet's Collision event with obj_enemy instead - the one place that actually represents a hit, targeting obj_gamemanager, the dedicated manager object Lesson 6 introduced for exactly this kind of shared state.

Level 1 - Full Worked Solution



These two checks look similar - both end with "the enemy is gone" - but only the hit check is allowed to touch enemies_remaining. Attaching the counter to a generic "was destroyed" hook instead of this specific hit check is exactly the bug GameMaker and 001 Game Creator have to fix this lesson.

The general escape-vs-hit distinction behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

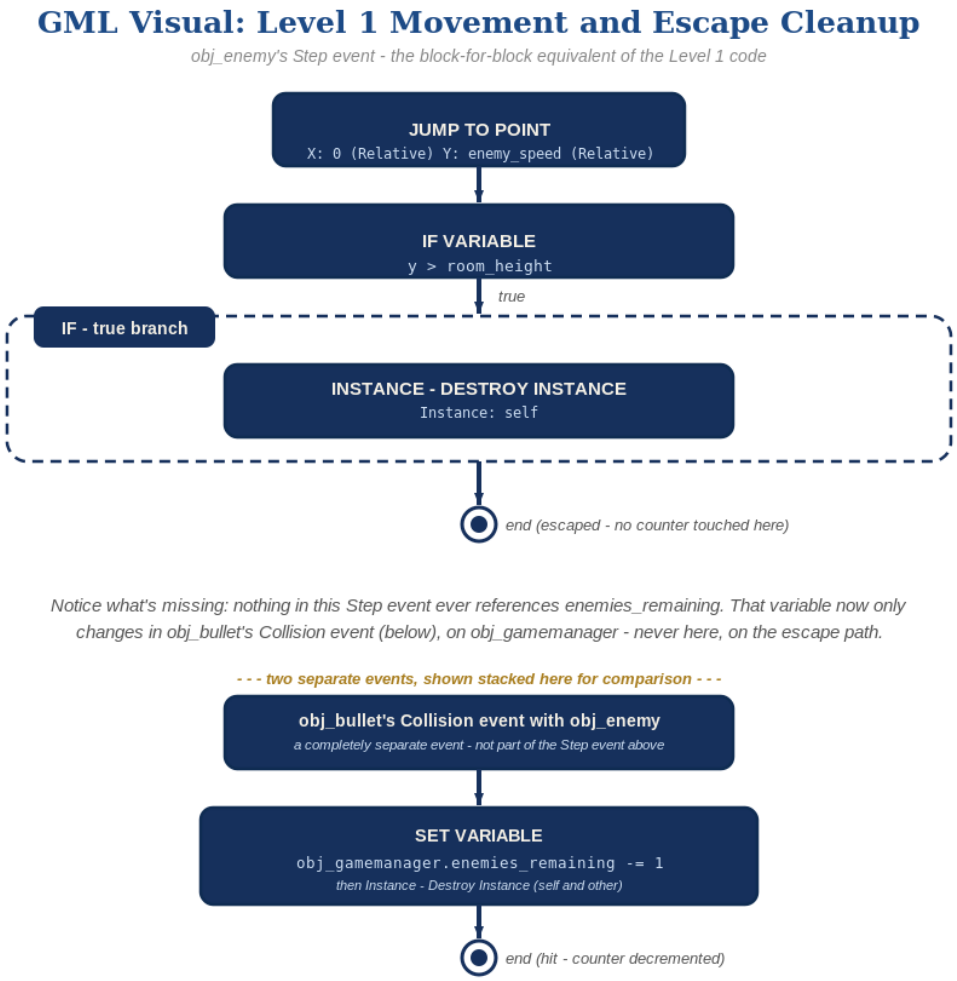
- obj_enemy's Step event handles movement (y += enemy_speed) and escape cleanup (instance_destroy() when y > room_height) - a plain destroy with zero awareness of win-counting.
- obj_bullet's Collision event with obj_enemy is now the single source of truth for obj_gamemanager's enemies_remaining, right alongside the sound effect and the two instance_destroy() calls already there from Lesson 4/6.
- obj_enemy's Destroy event is now empty (or deleted outright) - it has no job left to do.

```
// obj_enemy Create event:
enemy_speed = 1;

// obj_enemy Step event:
y += enemy_speed;
if (y > room_height) {
    instance_destroy();
}

// obj_bullet Collision event with obj_enemy:
audio_play_sound(sfx_hit, 1, false);
obj_gamemanager.enemies_remaining -= 1;
```

```
if (obj_gamemanager.enemies_remaining <= 0) {
    obj_gamemanager.has_won = true;
}
instance_destroy(other);
instance_destroy(self);
```

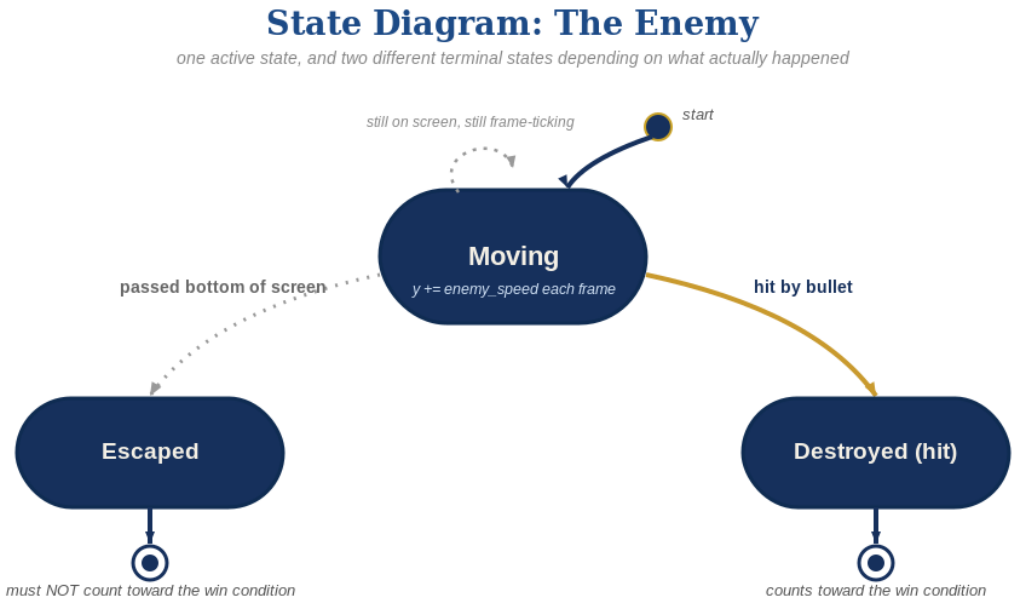


obj_enemy's Step event and obj_bullet's Collision event, in GML Visual - the block-for-block equivalent of the Level 1 code and the win-counter fix.

Why This Structure

- GameMaker's Destroy event is a lifecycle hook, not a gameplay event - it fires on ANY instance_destroy(), which is exactly why it was safe to use in Lesson 6 (only one destruction path existed) and exactly why it breaks in Lesson 7 (a second path was added).
- Collision events are inherently specific - obj_bullet's Collision event with obj_enemy can only ever fire when those two specific instance types actually overlap, which is a much stronger and more accurate signal for 'a kill happened' than 'something destroyed an enemy.'
- This mirrors the exact same principle taught with Scratch's per-clone touching check in Lesson 5 and Godot/Unity's signal-vs-direct-call distinction above: attach state-changing

logic to the specific event that means what you think it means, not to a generic lifecycle hook that fires for unrelated reasons too.



Once an enemy leaves Moving, it never comes back - both destinations are final. The whole trick this lesson teaches is making sure your code can tell these two final states apart, since only one of them is a kill.

The enemy's two possible endings this week - only one of them should count as a kill.

Level 2 / Level 3 Talking Points

- This is a good lesson to spend extra teacher-led time on, since it's a genuine bug students would have shipped in Lesson 6 without knowing it (the bug was latent - correct as of Lesson 6, broken only once Lesson 7's second destruction path exists).
- Ask students to predict, before testing: 'if we only fixed the Collision event but left the old Destroy event decrement in place, what would happen?' (Answer: double-counting - an enemy that's shot would decrement enemies_remaining twice, once from each event, causing the win condition to trigger after only half the enemies are actually destroyed.)
- Level 3 structure talking point: obj_enemy's Step event should never need to know enemies_remaining exists at all - if a student's Step event references it, that's a sign the fix wasn't fully applied.

Common Student Mistakes

- Leaving the old decrement in obj_enemy's Destroy event AND adding the new one to obj_bullet's Collision event - causes double-counting (see talking point above). This is the single most likely mistake this lesson; test for it explicitly.
- Moving the decrement to the Collision event but forgetting to delete/empty the Destroy event.
- Placing the escape-cleanup instance_destroy() in the Collision event instead of the Step event (unrelated concerns merged back together).